

Java Persistence API (JPA) + Hibernate

The **Java Persistence API (JPA)** is a specification that defines how to persist data in Java applications.

Object-Relational Mapping (ORM) is the process of converting Java objects to database tables. In other words, this allows us to interact with a relational database without any SQL.

Hibernate is one of the most popular Java ORM frameworks in use today. Additionally, Hibernate is a standard implementation of the JPA specification, with a few additional features that are specific to Hibernate.

Spring Data JPA is a Spring project that sits on top of JPA (typically Hibernate) to simplify database access. Define repository interfaces, and Spring generates queries automatically.

JDBC (Java Database Connectivity) is a low-level Java API to connect to relational databases. Hibernate generates SQL query and executes it using JDBC.

Setting Up JPA with Hibernate for MySQL(spring-boot):

↳ Dependencies

- `spring-boot-starter-data-jpa` gives JPA + Hibernate + Spring Data support
- `mysql-connector-j` lets app talk to a MySQL database

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

↳ Running MySQL on Docker:

```
db-mysql:
  image: mysql:8.3.0
  container_name: db-mysql
  environment:
    MYSQL_ROOT_PASSWORD: mysql
  ports:
    - "3307:3306"
  volumes:
    - ../data/mysql_db:/var/lib/mysql
    - ../docker/mysql/init_2.sql:/docker-entrypoint-initdb.d/init_2.sql
```

docker/mysql/init_2.sql file:

```
CREATE DATABASE IF NOT EXISTS e_commerce;
```

↳ application.properties file:

```
#Application
spring.application.name=db_performance

#Database
spring.datasource.url=jdbc:mysql://localhost:3307/e_commerce
spring.datasource.username=root
spring.datasource.password=mysql
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
server.port=8088

#db
# Automatically create database tables based on JPA entities.
# 'create' drops and recreates tables on each app start (good for dev, not prod).
# Use 'update' to apply changes without dropping existing data.
spring.jpa.hibernate.ddl-auto=create

# Show the generated SQL statements in the console/logs (helpful for debugging).
spring.jpa.show-sql=true

# Format the SQL output to be more readable with proper indentation.
spring.jpa.properties.hibernate.format_sql=true

# Enable Hibernate to collect and generate detailed statistics about its operations (queries, cache hits, etc.)
spring.jpa.properties.hibernate.generate_statistics=true

# Set logging level for Hibernate statistics to DEBUG to output detailed stats in the logs
logging.level.org.hibernate.stat=DEBUG

# Enable SQL query logging in console
logging.level.org.hibernate.SQL=DEBUG
```

↳ Creating Entity:

```
@Entity
@Table(name = "users") // 'user' is a reserved keyword in many DBs
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }
}
```

Explanation of Annotations:

- `@Entity` Marks the class as a JPA entity (a table in the DB).
- `@Table(name = "users")` Specifies the table name (avoids reserved keyword conflicts).
- `@Id` Indicates the primary key field.
- `@GeneratedValue` Defines how the primary key is generated (auto-increment).
- `@Column` Customizes column properties like uniqueness and length.
- `@PrePersist` Sets the timestamp just before saving the entity.

Lombok annotations (`@Getter` , `@Setter` , etc.) generate boilerplate code (getters/setters, constructors).

Some other annotations:

- **`@OneToOne`**
Defines a one-to-one relationship between two entities.
- **`@OneToMany`**
Defines a one-to-many relationship from one entity to many entities.
- **`@ManyToOne`**
Defines a many-to-one relationship where many entities reference one parent.
- **`@ManyToMany`**
Defines a many-to-many relationship between entities.
- **`@JoinColumn(name = "...")`**
Specifies the foreign key column used in a relationship.
- **`@JoinTable(name = "...")`**
Specifies the name of the join table used to manage many-to-many relationships between two entities.
- **`@MappedSuperclass`**
Marks a class whose fields are inherited by entities but doesn't create a table.
- **`@Embeddable`**
Marks a class as a value object that can be embedded into an entity.
- **`@Embedded`**
Embeds a value object marked with `@Embeddable` into an entity.
- **`@Enumerated(EnumType.STRING)`**
Stores an enum as a string value in the database.
- **`@Lob`**
Marks a field to be stored as a large object (CLOB or BLOB).
- **`@Transient`**
Excludes a field from persistence (not stored in the database).
- **`@Version`**
Enables optimistic locking by tracking entity versions for concurrency control.
- **`@NamedQuery(name = "...", query = "...")`**
Defines a reusable JPQL query with a name.

Lifecycle Callbacks:

- Besides `@PrePersist`,
- `@PreUpdate` — before update
- `@PostPersist` — after insert
- `@PostLoad` — after loading from DB
- `@PreRemove` — before delete
- `@PostRemove` — after delete

These let you add logic during entity lifecycle events.

Here's how to use the `User` entity with a Spring Data JPA repository:

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
    User findByEmail(String email);
}
```

This interface can now be used within the service to save data to the database or read data.

```
@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User createUser(User user) {
        return userRepository.save(user);
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public Optional<User> getUserById(Long id) {
        return userRepository.findById(id);
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }
}
```

Now that we know the basic setup, let's deep dive into Hibernate.

Hibernate Caching: First-Level, Second-Level, and Query Cache

Hibernate caching improves application performance by reducing the number of database queries. It stores entities or query results temporarily in memory so that repeated access can be served faster.

Hibernate provides **three levels of cache**:

- First-Level Cache (Session Cache): Per Hibernate Session (i.e., per transaction). Enabled By Default.
- Second-Level Cache (SessionFactory Cache): Shared across Sessions within a SessionFactory. Not Enabled By Default.
- Query Cache: Caches the results of HQL/JPQL or Criteria queries. Depends on the **second-level cache** (only cached entities can be queried). Not enabled by default; must be configured separately.

Cache Eviction Strategies and Pitfalls:

Eviction Strategies

- Time-based eviction: Automatically remove cache entries after a TTL (time-to-live).
- Size-based eviction: Remove the least recently used entries when the cache size exceeds a limit.
- Manual eviction: Explicitly evict or clear caches programmatically when data changes.

Common Pitfalls:

- Stale data: Cache might serve outdated data if eviction policies aren’t tuned or updates aren’t synchronized.
- Memory Overuse: Large cache sizes can lead to memory exhaustion.

Hibernate’s Fetch Strategies: Eager vs Lazy Loading

When using Hibernate (via JPA) in Spring Boot, fetch types determine when related entities are loaded from the database.

Hibernate provides two main fetch strategies:

1. EAGER — load related data immediately along with the main entity.
2. LAZY — load related data only when accessed (on-demand).

Key Characteristics:

Aspect	Eager Loading	Lazy Loading	
When Loaded	Immediately with the parent entity	On-demand when accessed	
Default For	@ManyToOne , @OneToOne	@OneToMany , @ManyToMany	
Performance	Higher initial query cost	Lower initial cost, but risk of N+1 queries	
Annotation	fetch = FetchType.EAGER	fetch = FetchType.LAZY	
Use Case	Small, always-needed relationships	Large or optional relationships	

Example:

Lazy Loading

```
@Entity
public class Order {
    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
    private List<OrderItem> items;
}
```

- When you fetch an `Order`, the `items` list is not loaded until you explicitly call `order.getItems()`.
- Useful when the relationship is large and not always needed.

Eager Loading

```
@Entity
public class User {
    @OneToOne(fetch = FetchType.EAGER)
    private Profile profile;
}
```

- When you fetch a `User`, Hibernate immediately fetches the `Profile` in the same query (or via an extra query, depending on the mapping).

How to Avoid `LazyInitializationException`:

`LazyInitializationException` occurs when lazy-loaded data is accessed outside an active Hibernate Session (e.g., after the transaction ends).

Here are strategies to prevent it:

1. Access inside a transaction:

Ensure all lazy fields are accessed within the service layer's transactional method.

```
@Transactional
public Order getOrderWithItems(Long id) {
    Order order = orderRepository.findById(id).orElseThrow();
    order.getItems().size(); // Force load
    return order;
}
```

2. Use Fetch Joins in JPQL/HQL:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.id = :id")
Order findOrderWithItems(@Param("id") Long id);
```

3. `@EntityGraph` & fetch joins :

`@EntityGraph` allows you to define which associations to fetch eagerly, overriding default fetch types.

```
public interface UserRepository extends JpaRepository<User, Long> {
    @EntityGraph(attributePaths = {"addresses"})
    Optional<User> findById(Long id);
}
```

- Behavior: Fetches the User and its addresses in a single query, ignoring `FetchType.LAZY`.

Fetch joins in JPQL or Criteria API, explicitly load associations in a single query.

```
// JPQL Example:
public interface UserRepository extends JpaRepository<User, Long> {
    @Query("SELECT u FROM User u JOIN FETCH u.addresses WHERE u.email = ?1")
    User findByEmailWithAddresses(String email);
}
```

- Fetches User and addresses in one query, avoiding `LazyInitializationException`.

N+1 Problem:

The N+1 query problem occurs when Hibernate executes one initial query to fetch a list of entities (the “1”), followed by N additional queries to load their lazy associations (the “N”).

To Avoid This N+1 Problem:

- Use Fetch Joins for Eager Loading in Queries.
- Leverage `@EntityGraph` for Fine-Grained Fetching
- Enable Batch Fetching

Hibernate & Transactions: Managing Data Consistency in Spring Boot

1. The Role of Transactions in JPA

A transaction is a unit of work that ensures data consistency — either all operations succeed (commit) or none take effect (rollback).

Key benefits:

- Atomicity — Ensures all operations (e.g., inserting a User and its Addresses) succeed or fail together.
- Consistency — Maintains database rules, like constraints or business logic (e.g., unique email in User).
- Isolation — Transactions don’t interfere with each other. Controls how concurrent transactions interact, preventing issues like dirty reads.
- Durability — Once committed, changes are permanent.

Example: Saving `User` and `Address` together atomically.

```

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;
    @Autowired
    private AddressRepository addressRepository;

    @Transactional
    public void createUserWithAddress(User user, Address address) {
        userRepository.save(user);

        // If this fails (e.g., constraint violation), the whole transaction rolls back
        addressRepository.save(address);
    }
}

```

If `addressRepository.save(address)` fails, the `user` insert is also rolled back.

2. Understanding Transaction Propagation:

- `REQUIRED` (*default*) - Join existing transaction or create a new one if none exists.
- `REQUIRES_NEW` - Always start a new transaction, suspending the current one.
- `MANDATORY` - Must run inside an existing transaction, else throw an exception.
- `SUPPORTS` - Run within a transaction if one exists, else non-transactional.
- `NOT_SUPPORTED` - Suspend any existing transaction and run non-transactionally.
- `NEVER` - Must not run inside a transaction; throws exception if one exists.
- `NESTED` - Run within a nested transaction (can roll back independently).

Example:

```

@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    @Transactional(propagation = Propagation.REQUIRED) // default
    public void placeOrder(Order order) {
        // Save order
        // Calls payment
        paymentService.processPayment(order.getId());
    }
}

@Service
public class PaymentService {

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void processPayment(Long orderId) {
        // Always runs in a new transaction
        // Even if placeOrder fails, this transaction is independent
    }

    @Transactional(propagation = Propagation.NESTED)
    public void logPayment(Long orderId) {
        // Nested transaction can rollback independently
    }
}

```



```
    }  
}
```

3. Isolation Levels:

Isolation levels prevent concurrency issues like

- dirty reads (reading uncommitted data),
- non-repeatable reads (data changes mid-transaction), or
- phantom reads (new rows appear mid-transaction).

Set via `@Transactional(isolation = Isolation)` . Default is database-specific (often `READ_COMMITTED`).

The highest Level of Isolation is **Serializable** , it makes transactions run one after another as if they were in a single line. This provides strong consistency. But this could really slow things down as they need to wait for their turn.

To speed things up, databases often provide lower isolation levels like `REPEATABLE_READ` , `READ_COMMITTED` , `READ_UNCOMMITTED` .

Isolation Level	Description	Concurrency Issues Prevented	Performance Impact
READ_UNCOMMITTED	Allows dirty reads.	None (lowest isolation).	Highest performance, but risky.
READ_COMMITTED (common default)	Prevents dirty reads.	Dirty reads.	Good balance.
REPEATABLE_READ	Prevents dirty and non-repeatable reads.	Dirty and non-repeatable reads.	Moderate locking overhead.
SERIALIZABLE	Prevents all issues, like running serially.	Dirty, non-repeatable, and phantom reads.	Lowest performance due to heavy locking.

Isolation Levels

Example:

```
@Service  
public class BankService {  
  
    @Autowired  
    private AccountRepository accountRepository;  
  
    @Transactional(isolation = Isolation.SERIALIZABLE)  
    public void transfer(Long fromId, Long toId, double amount) {  
        Account from = accountRepository.findById(fromId).orElseThrow();  
        Account to = accountRepository.findById(toId).orElseThrow();  
  
        from.debit(amount);  
        to.credit(amount);  
  
        accountRepository.save(from);  
        accountRepository.save(to);  
    }  
}
```

- With `Isolation.SERIALIZABLE` Each transfer transaction executes strictly in order (like a queue).
- For faster performance but less strict guarantees, use `READ_COMMITTED` or `REPEATABLE_READ`.

4. Using `@Transactional` Effectively:

- Annotate at Service Layer: Place on public service methods, not repositories (Spring Data JPA handles repo transactions internally).
- Specify Read-Only: Use `@Transactional(readOnly = true)` for queries to optimize (e.g., disable dirty checking).
- Set Timeout: `@Transactional(timeout = 30)` (seconds) to prevent long-running transactions.
- Avoid on Private Methods: `@Transactional` works only on public methods due to proxying.
- Combine with Propagation/Isolation: As needed for specific scenarios.

Example:

```
@Service
public class ProductService {

    @Autowired
    private ProductRepository productRepository;

    // Read-only transaction: skips dirty checking
    @Transactional(readOnly = true)
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    // Timeout example
    @Transactional(timeout = 10) // seconds
    public void slowOperation() throws InterruptedException {
        Thread.sleep(15000); // will throw TransactionTimedOutException
    }
}
```

5. Handling Rollback and Exception Scenarios:

Transactions rollback on unchecked exceptions (e.g., `RuntimeException`) by default. Checked exceptions (e.g., `Exception`) commit unless specified.

Rollback Strategies:

- Default Behavior: Rollback on `RuntimeException` or `Error`; commit on checked exceptions.
- Customize: Use `@Transactional(rollbackFor = {CustomException.class})` to rollback on specific exceptions, or `noRollbackFor` to exclude.
- Manual Rollback: Inject `TransactionStatus` and call `setRollbackOnly()`.
- Exception Handling: Wrap in try-catch, but let exceptions propagate for automatic rollback.

Common Scenarios:

- Partial Failure: If saving a User succeeds but Address fails (e.g., constraint violation), the whole transaction rolls back.

- **Optimistic Locking:** Hibernate throws `OptimisticLockException` on version conflicts; handle with retries.
- **Pessimistic Locking:** Use `@Lock(LockModeType.PESSIMISTIC_WRITE)` for explicit locks in high-concurrency scenarios.

Examples:

Handling Rollback & Exceptions:

```
@Service
public class BookingService {

    @Autowired
    private BookingRepository bookingRepository;

    // Rollback for RuntimeException (default)
    @Transactional
    public void bookFlight(String user) {
        bookingRepository.save(new Booking(user));

        if (true) throw new RuntimeException("Payment failed");
        // Entire transaction rolled back
    }

    // Rollback for checked exceptions explicitly
    @Transactional(rollbackFor = {Exception.class})
    public void bookHotel(String user) throws Exception {
        bookingRepository.save(new Booking(user));

        if (true) throw new Exception("Hotel system failed");
        // Rollback happens because we declared rollbackFor
    }

    // No rollback for specific exception
    @Transactional(noRollbackFor = {IllegalArgumentException.class})
    public void bookCar(String user) {
        bookingRepository.save(new Booking(user));

        throw new IllegalArgumentException("Minor issue, don't rollback");
        // Transaction commits despite exception
    }
}
```

Optimistic Locking:

```
@Entity
public class Product {
    @Id
    private Long id;

    private String name;

    private int stock;

    @Version // Enables optimistic locking
    private int version;
}
```

```
@Service
public class InventoryService {

    @Transactional
    public void updateStock(Product product, int qty) {
        product.setStock(product.getStock() - qty);
        // If another transaction updates same row, Hibernate throws OptimisticLockException
    }
}
```

```
}  
}
```

Pessimistic Locking:

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
  
    @Lock(LockModeType.PESSIMISTIC_WRITE)  
    @Query("SELECT p FROM Product p WHERE p.id = :id")  
    Product findByIdForUpdate(@Param("id") Long id);  
}
```

```
@Service  
public class WarehouseService {  
  
    @Autowired  
    private ProductRepository productRepository;  
  
    @Transactional  
    public void reserveProduct(Long productId) {  
        Product product = productRepository.findByIdForUpdate(productId);  
        // Locks row until transaction ends  
        product.setStock(product.getStock() - 1);  
    }  
}
```

Don't manually code “growing” or “shrinking” phases — the **DB transaction manager enforces 2PL** when you use pessimistic locking.

Note:

If we have **one instance of a JVM** (single process, multithreaded app like a Spring Boot service), concurrency control means **synchronizing access to shared in-memory resources** so multiple threads don't corrupt data.

Ways to Control Concurrency in JVM:

- synchronized

```
public synchronized void updateBalance() {  
    // only one thread at a time  
}
```

- ReentrantLock (Explicit Lock)

```
private final ReentrantLock lock = new ReentrantLock();  
  
public void updateBalance() {  
    lock.lock();  
    try {  
        // critical section  
    } finally {  
        lock.unlock();  
    }  
}
```

```
        }
    }
}
```

The Problem with @ManyToMany + List in Hibernate

When using a `List` for a `@ManyToMany` relationship, Hibernate automatically maintains an **order index** column in the join table to preserve element ordering.

Example:

user_id	role_id	order_index
1	2	0
1	3	1
1	4	2

What Happens on Delete:

If you remove one element from the middle of the list:

- 1. Hibernate deletes that row from the join table.
- 2. It then updates the `order_index` for **all subsequent rows** to keep the sequence correct.

This means a single removal can cause:

- **1 DELETE + (N-1) UPDATES** for N elements after the removed one.

Extra Inserts on Reorder:

If the order of elements changes:

- Hibernate may delete all join table entries for that entity.
- Then re-insert them in the new order — causing multiple DELETE and INSERT statements.

Why It’s a Problem:

- **Performance overhead:** Many unnecessary JDBC statements.
- **Lock contention:** Updates lock more rows than necessary.
- **Wasted I/O:** The database does extra work.

Best Practices:

- Prefer a `Set` for `@ManyToMany` relationships unless ordering is truly required.
- If ordering is essential, map the join table as a separate entity to control updates manually.